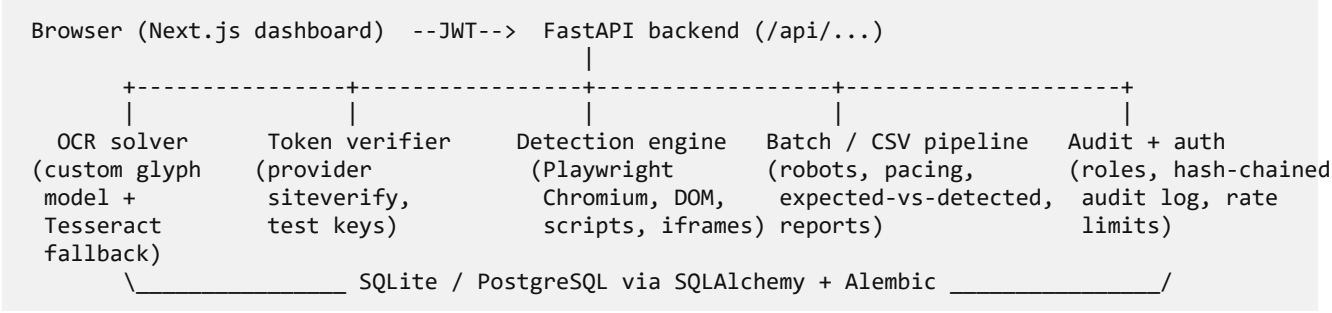
Architecture and Cost Breakdown

CAPTCHA Solving & Verification Tool · short explanation for reviewers

1. Architecture



Part	What it does
Recognition	Classifies a challenge as synthetic text, or as Google reCAPTCHA / hCaptcha / Cloudflare Turnstile (subtype and widget state); anything else is reported as **unsupported** and sent to human review.
Solving engine (custom)	Our own synthetic text CAPTCHAs are solved by segmentation plus a NumPy glyph classifier. **Hybrid** routing falls back to tuned Tesseract when the custom model is unsure. Each result has the prediction, a confidence value, the engine route and the time taken. Held-out accuracy: **99.3% exact match on 300 images** (same generator; see limits).
Fallback	Custom model first, Tesseract second (hybrid). A fallback reason is returned in the response.
Token verification	Server-side POST to each provider's official `siteverify` endpoint with the providers' published test keys (reCAPTCHA, hCaptcha, Turnstile). Handles invalid, expired, reused and missing tokens, wrong secret, provider down and malformed replies. Returns a structured response; tokens and secrets are never logged or stored.
Reusable API	`POST /api/captcha/ocr-demo`, `/ocr-benchmark`, `/verify-token`, `/classify` (JWT required; Swagger at `/docs`).
Web interface	OCR Laboratory, Verification, URL Inspector, Scan History, CAPTCHA Results, Human Review, CSV Batch Manager, Reports, Evaluation (attempts, successes, failures, processing time, challenge type, method).
Reliability and safety	Timeouts on every external call, per-host pacing, robots.txt respected, domain allow-list, SSRF guard, rate limits, role-based access, tamper-evident audit trail.

2. Cost breakdown

Item	Cost
Third-party CAPTCHA-solving API	**None used - $0.** No paid solver is integrated, so there is no per-challenge fee, pricing table or monthly cap to document.
Custom OCR, per challenge	Local CPU only. Measured mean latency: hybrid **5.7 ms** (p95 8.3 ms), custom model 4.5 ms, tuned Tesseract 246.7 ms. Cost per challenge = (latency in ms / 3,600,000) × the price of one CPU-core-hour of your machine; no hourly price is assumed here.

Provider verification calls	Demo uses the providers' free published **test** keys: no charge. With real production keys, each provider's own terms and pricing would apply; that was not researched or assumed.
Training (one-off)	About 55 s of CPU for 60 epochs on about 14,900 glyphs.
Software licences	All dependencies are open source (for example Tesseract is Apache-2.0).
Hosting	None performed, so no hosting cost is claimed.

3. Operational limits and error handling

Limit	Behaviour
Provider timeout	8 s per verification call (`VERIFY_TIMEOUT_SECONDS`); no automatic retries, so an outage cannot multiply calls.
Rate limits	Per-user limits on compute, benchmark, login and admin routes (configurable); HTTP 429 with Retry-After.
Input limits	Maximum image size and pixels, JSON body size and CSV size are enforced.
Scope of solving	Only our own synthetic text CAPTCHAs are solved. Production reCAPTCHA, hCaptcha and Turnstile challenges are identified and routed to a human; they are not bypassed.
Accuracy claim	The 99.3% figure applies to our own generator's images and is not a claim about real-world CAPTCHAs.

4. Setup (summary)

Python 3.12+, Node 20+, Tesseract 5. Create a venv, install `requirements.txt` , run `playwright install chromium` , copy `.env.example` to `.env` (set `JWT_SECRET_KEY` and `AUDIT_HMAC_KEY`), run `alembic upgrade head` , create the first admin with `python -m app.cli create-admin` , start the backend with uvicorn and the frontend with `npm run build && npm run start` . Full steps and the environment variable table are in the repository README.